

Kailo Coaching Hub Audit

Product review & data-platform architecture

21 August 2026

Defining Product Goals

- **Team-native, coach-facing training platform.** The team is the unit; the coach is the primary user.
- **Stronger analytical tools and coach customization** than TrainingPeaks or FinalSurge — the analytics and the ability to tailor the surface are the differentiators, not feature parity.
- **A coach-determined path for athletes to interact** that *reduces* the need for athlete buy-in. The less work an athlete has to do, the better the product: participation should be near-passive by default, with the coach deciding how much the athlete is asked to do.

Technical Needs

- **Athlete objects with histories and attributes.** Each athlete carries a history (race and training paces over time) *and* a set of learned attributes — e.g. “performs better on hilly courses,” “degrades more in rain / heat.” These attributes are estimated from data, they change as more data arrives, and the set will grow. The data platform exists to populate and keep refreshing them.

1 Product Audit

1.1 Organization

The current coaching hub (desktop) is organized on two axes: a **thematic vertical axis** and a **temporal horizontal axis**. The vertical axis is a left pane on wide screens, moving to the top on narrow screens. None of its top-level labels are clickable. Its structure:

Before Practice	After Practice	The Roster
Today	Session review	Week wall
Tasks + Links	Inbox	Volume
Planner		Athlete
Workout planner		Parameters
Pace cards		Results
Share with the Team		Predictions
Athlete day		
Team board	Housekeeping	
1-on-1s	Operations	
	Data health	
	How this works	
	Setup	

The landing page (kailo.fit/hub, “Today”)

The landing page is a long stack of text widgets. Top widgets include “[N] uploaded yesterday,” “Available today,” “Data flags,” “Gone quiet,” “Need a look,” and “Dew point at practice.” Below these sit more text widgets: “Where your attention goes” (activities flagged as abnormal — average heart rate high on an easy run, distance longer than expected), a “Today’s plan” widget with a print option, and a practice conditions widget (“Conditions this afternoon”).

Specific notes:

- **Keep high-quality, accessible print.** The plan’s print option is good and should stay easy to reach throughout.
- **Conditions is buried and single-window.** “Conditions this afternoon” sits too far down, does not currently populate, and only shows the afternoon — athletes running *doubles* lose the morning window entirely.
- **No expand/collapse** on any widget, producing a very long page.

Assessment. There is too much information, almost all of it text, with no visual hierarchy to separate what matters from what does not. Because this is the landing page, deep customization is essential — and this critique of the landing page is the critique of the whole product.

1.2 Proposed structure: five main tabs

Collapse the sprawl into five tabs.

1. **Today** — weather for *four* windows: this morning, this afternoon, next morning, next afternoon (fixing the doubles gap).
2. **Upcoming.**
3. **Athletes** — see the athlete-page layout below.
4. **Recruiting.**
5. **Settings.**

The athlete page (three tiers, top to bottom)

Identity (top, glanceable).

Name, gender, class, hometown, MileSplit and Athletic.net links, and PRs.

Secondary.

Attendance / training log, past and current injuries, upcoming registered events (date, name, event), training paces.

Advanced analytics (bottom).

The learned attribute models:

- **Hill Resilience** — is the runner better on flat or hilly courses? Built from course-elevation learning, MileSplit course rankings, FloRatings A/B/C/D, TullyRunners speed ratings, joshm.dev’s Crystal Springs Pace Factor, and the golf-slope-rating analog.
- **Weather Impact** — how much adverse weather degrades this runner. The constraint: exact per-race time is needed, and getting it at scale from Athletic.net is not viable (see §2).
- Additional analytical models as desired.

1.3 Wording

Much of the copy reads as too cute, too artsy, or too AI-generated, and should be plainer. Examples:

Current	Plainer
“One line from an athlete at 3:47 is worth more than finding out at 4:10 in the parking lot.”	Cut, or state the actual point.
“Where your attention goes” — “ranked, and every name is a door.”	“Needs a look,” with names that link.
“Yesterday, rolled up”	“Yesterday, summarized.”

2 Data and Enrichment Platform

The athlete attributes in §1 (hill resilience, weather impact, and the models to come) are only as good as the data feeding them. This section lays out how that data is sourced, enriched, and linked — and why the platform must **divest fully from Athletic.net** as a scale dependency.

2.1 The Athletic.net problem

Athletic.net has the broadest US high-school coverage, and the project was built on it. But it is not a trustworthy dependency at scale:

- **It blocks us.** Cloudflare bot management returns `cf-mitigated=challenge` on sustained crawling. Measured over weeks: $\leq 1\text{--}2$ req/s ran clean for days; 3 req/s drew 24-hour blocks on two consecutive days; volume near 30,000 requests/day drew challenges regardless of rate. Exit-node rotation across six IPs did not prevent it — the block is on the traffic *pattern*, not the address.
- **No sanctioned access.** No API, no developer program, no data-partnership channel.
- **The data itself has defects** we must work around — relay results collapse to one athlete per squad; per-race timestamps are all `T00:00:00`, so it cannot supply the race times the Weather Impact model needs anyway.

Assessment. Athletic.net stays as a redundancy check and for the states already captured, but every *new* pipeline is designed against sources that answer reliably — TFRRS, MileSplit, and static timing archives. The three diagrams below are that design.

2.2 (1) Roster, meet, and team information

How it works, per source.

- **TFRRS (college).** Seeded from the seven NCAA-division region pages, walk teams $\rightarrow$ seasons $\rightarrow$ meets. A single meet *event* page carries the whole field (place, athlete, class year, team, mark), so one fetch covers every school in a race. Corpus: 8.1M rows, all three divisions, XC + indoor + outdoor, 2021–2026.
- **MileSplit (HS).** `/teams?state=xx` then the proven per-team, per-athlete ingest path, smallest state first. Polite by default (rate-limited, ledger tee). Note the enumeration cap: `/teams?state` returns at most 999 rows, so the largest states need a per-county or meet-harvest route for full coverage.
- **Divest path.** New states come from MileSplit; existing Athletic.net captures (CA/TX/FL/NY) are retained and merged, but no new crawl depends on that host.

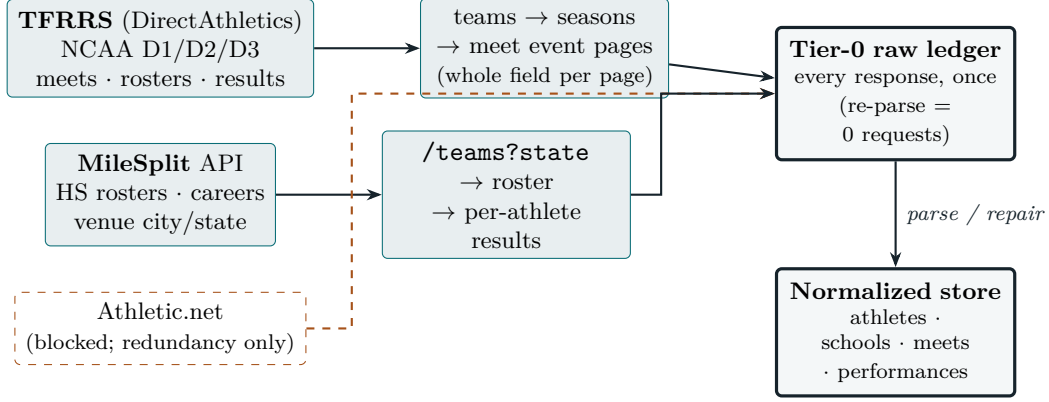


Figure 1: Roster/meet/team sourcing. Every response tees to a raw ledger before parsing, so a parser fix replays at zero request cost. Athletic.net (dashed) is demoted to redundancy.

2.3 (2) Course weather and hill information

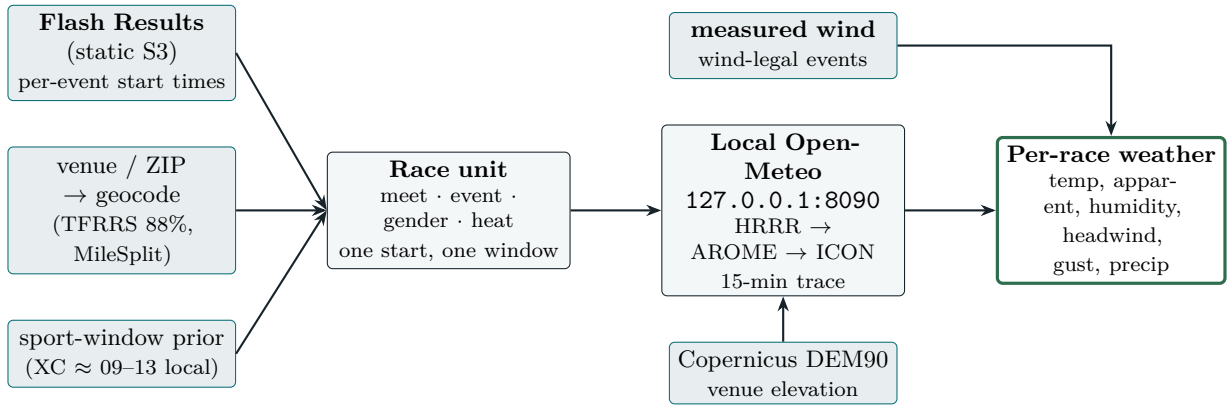


Figure 2: Weather + elevation per *race*, not per meet. The 11 am women’s JV and the 3 pm men’s varsity get different windows. Wind for sprints comes measured from the results themselves.

Hill / course difficulty (three complementary signals):

1. **Empirical ALS course offsets (built).** A two-way log-time model — $\log(\text{equiv5k}) = \text{ability} + \text{offset}$, i.e. an athlete-season term plus a course term — solved by alternating least squares over athletes who race multiple courses. Validated split-half $r = 0.98$; recovered Crystal Springs (+72s) and Woodbridge (−50s) with no external input. It measures what terrain *costs*, but bundles hills with footing and weather.
2. **External course ratings (cross-check + decomposition).** TullyRunners speed ratings, MileSplit course rankings, FloRatings A/B/C/D, joshm.dev Crystal Springs Pace Factor, golf-slope analog. Joining these and correlating with the ALS offset validates our number and separates terrain-driven from conditions-driven difficulty.
3. **True elevation gain (targeted).** GPS/GPX course traces through the DEM give cumulative climb per km — the only physical hill metric, but per-course external data. Reserve for marquee courses.

Assessment. Once weather is attached per race, re-fit the ALS *net of the weather term*: the course offset then stops absorbing that day’s heat and becomes a terrain-only score — the cleanest hill metric derivable without a GPS trace.

2.4 (3) Linking college and high-school runners

There is no crosswalk: zero athletes carry both an Athletic.net id and a TFRRS id, and MileSplit’s `tfrsId` field is null on every bio examined. Linkage is therefore *probabilistic*, and the honest deliverable is the ceiling.

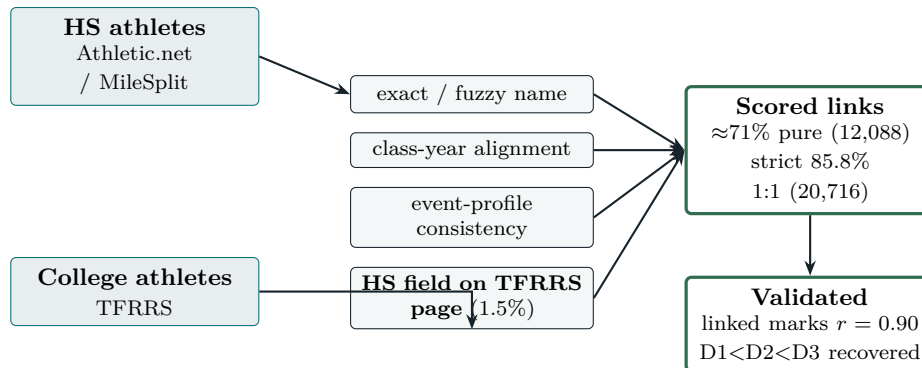


Figure 3: Probabilistic HS→college linkage. No exact key exists; a scored combination of name, year and event profile reaches ~71% precision, and TFRRS athlete pages carry a literal High School field (1.5% of pages) that opens a path to a *real* crosswalk by crawling those pages directly.

The route to a real crosswalk. TFRRS athlete pages carry a literal “*High School:*” field on 1.5% of archived pages (roster pages do not). Crawling athlete pages directly — rather than inferring identity from names — upgrades the linkage from probabilistic to keyed for the athletes who have it, and is the recommended next step for the recruiting and progression models.

2.5 Build order

Steps that cost nothing against any host come first:

1. Venue + geo columns; reparse venue from the archived TFRRS/MileSplit payloads. **Zero requests.**
2. Wind column on performances; backfill from archived Athletic.net payloads (wind-legal events). **Zero requests.**
3. Flash Results time crawler — new host, static S3, its own polite profile and ledger. Start with the NCAA championship/regional set and measure the match rate before scaling.
4. Weather adapter: emit a per-race manifest, run the existing `~/weather` Open-Meteo pipeline unchanged, reduce to race-window aggregates.
5. Course difficulty: ship the ALS offset, join external ratings as a check, then re-fit net of weather for a terrain-only score.